

Candidate Sampling

Dong Bingfeng

<http://ml.dongbingfeng.cn>

Motivation

- A great disparity in count between positive and negative samples with limited computational resources.
 - CTR/CVR prediction models.
 - Disease discrimination models.
- Softmax model with a large number of classifications.
 - Words prediction models in NLP.
 - DNN information retrieval models.

Odds

- Definition

$$\text{odds}(p \text{ vs } q) = \frac{p}{q}, \quad \text{odds}(p) = \frac{p}{1-p}$$

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = \text{logodds}(p)$$

- Odds and LR

$$p = \frac{1}{1 + \exp(\mathbf{w}^T \mathbf{x})},$$

$$\text{logodds}(p) = \text{logit}(p) = \mathbf{w}^T \mathbf{x}$$

Sampled Logistic

- We have sampled negative samples with distribution $Q(y|\mathbf{x})$, and wish to obtain $P(y|\mathbf{x})$

$$\begin{aligned}\mathbf{w}^T \mathbf{x} &= \text{logodds}(y \text{ came from Positive vs Negative} \mid \mathbf{x}) \\ &= \log \frac{P(y|\mathbf{x})}{Q(y|\mathbf{x})(1 - P(y|\mathbf{x}))} = \log \frac{P(y|\mathbf{x})}{1 - P(y|\mathbf{x})} - \log Q(y|\mathbf{x})\end{aligned}$$

With sampling ratio $1/r$, $Q(y|\mathbf{x}) = 1/r$

$$F(\mathbf{x}, y) = F'(\mathbf{x}, y) + \log Q(y|\mathbf{x}) = \mathbf{w}^T \mathbf{x} - \log(r)$$

Sampled Softmax

- Softmax **Training** Process

$$p(y|\mathbf{x}) = \text{softmax}(\mathbf{x}) = \frac{1}{Z} \exp(\mathbf{w}_y^T \mathbf{x}), \quad \text{where } Z = \sum_y \exp(\mathbf{w}_y^T \mathbf{x})$$

$$\nabla \log p(y|\mathbf{x}) = \nabla \mathcal{E}(y) - \sum_{y_k} p(y_k|\mathbf{x}) \nabla \mathcal{E}(y_k), \quad \text{where } \mathcal{E}(y_k) = \mathbf{w}_{y_k}^T \mathbf{x}$$

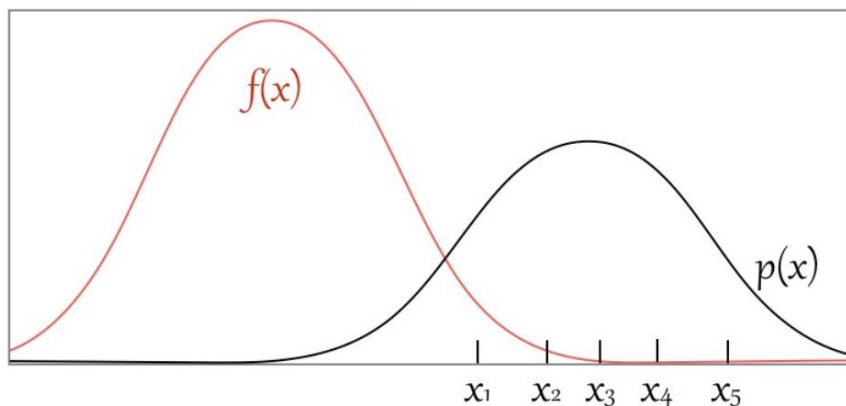
$$\sum_{y_i} p(y_i|\mathbf{x}) \nabla \mathcal{E}(y_i) = E_{y_i \sim P} [\nabla \mathcal{E}(y_i)] \simeq \sum_{y_k} \hat{w}(y_k) \nabla \mathcal{E}(y_k)$$

$$\text{where } \hat{w}(y_k) = \frac{\tilde{w}(y_k)}{\sum_{j=1}^m \tilde{w}(y_j)}, \quad \tilde{w}(y_k) = \tilde{p}(y_k) / \tilde{q}(y_k) = \mathbf{w}_{y_k}^T \mathbf{x} / \tilde{q}(y_k)$$

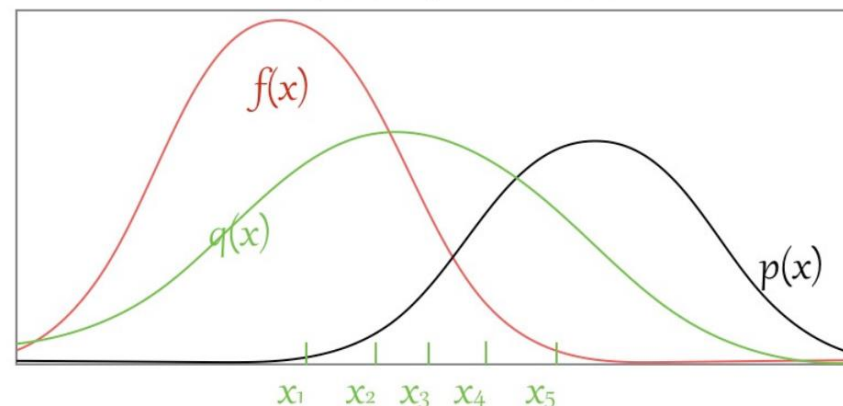
Importance Sampling

- We need to obtain $E[f(x)], x \sim p$

i.e.
$$E[f(x)] = \int_x f(x)p(x)dx = \int f(x)\frac{p(x)}{q(x)}q(x)dx = \int f(x)w(x)q(x)dx$$



(a)



(b)

https://blog.csdn.net/Dark_Scope

$$Ef(x) = \sum_{i=1}^m \hat{w}(x_i) f(x_i) \quad \text{where} \quad \hat{w}(x_i) = \frac{\tilde{w}(x_i)}{\sum_{j=1}^m \tilde{w}(x_j)} \quad x_i \sim q$$

Noise-Contrastive Estimation (NCE)

- For a statistical model which is specified through an unnormalized pdf $p_m^0(.; \alpha)$, we include the normalization constant as another parameter c of the model.

$$\text{i.e.} \quad \ln p_m(.; \theta) = \ln p_m^0(.; \alpha) + c$$

$X = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ is observed data set; $Y = (\mathbf{y}_1, \dots, \mathbf{y}_T)$ is an artificially generated data set of noise y with distribution $p_n(.)$

$$\hat{\theta}_T = \operatorname{argmax}_{\theta} J_T(\theta) \quad \text{where}$$

$$J_T(\theta) = \frac{1}{2T} \sum_t \ln [h(\mathbf{x}_t; \theta)] + \ln [1 - h(\mathbf{y}_t; \theta)]$$

$$h(\mathbf{u}; \theta) = \frac{1}{1 + \exp [-G(\mathbf{u}; \theta)]},$$

$$G(\mathbf{u}; \theta) = \ln p_m(\mathbf{u}; \theta) - \ln p_n(\mathbf{u}).$$

Theory of NCE

- Define pdf of positive samples: $p_d(x_i) = p_{d_i}$; pdf of negative samples: $p_n(x_i) = p_{n_i}$; pdf with parameter θ of samples: $p_m(x_i) = p_{m_i}$; N is the total samples number.
- Probability of observation

$$e^{l(\theta)} = \prod_i \left(\frac{p_{m_i}}{p_{m_i} + p_{n_i}} \right)^{N \cdot p_{d_i}} \left(\frac{p_{n_i}}{p_{m_i} + p_{n_i}} \right)^{N \cdot p_{n_i}}$$

$$\operatorname{argmax}_{\theta} l(\theta) = \operatorname{argmax}_{\theta} \sum p_{d_i} \ln \left(\frac{p_{m_i}}{p_{m_i} + p_{n_i}} \right) + p_{n_i} \ln \left(\frac{p_{n_i}}{p_{m_i} + p_{n_i}} \right)$$

- Derivative $l(\theta)$ by p_{m_i} , we get $p_{d_i} = p_{m_i}$ when maximize $l(\theta)$.
- Note that there is NO constriction of $\sum p_{m_i} = 1$, so p_{m_i} is normalized by default.

Theory of NCE

- **Theorem:** If conditions (a) to (c) are fulfilled then $\hat{\theta}_T$ converges in probability to θ^* , i.e. $\hat{\theta}_T \xrightarrow{P} \theta^*$.

(a) $p_n(\cdot)$ is nonzero whenever $p_d(\cdot)$ is nonzero

(b) $\sup_{\theta} |J_T(\theta) - J(\theta)| \xrightarrow{P} 0$

(c) $\mathcal{I} = \int \mathbf{g}(\mathbf{x})\mathbf{g}(\mathbf{x})^T P(\mathbf{x})p_d(\mathbf{x})d\mathbf{x}$ has full rank, where

$$P(\mathbf{x}) = \frac{p_n(\mathbf{x})}{p_d(\mathbf{x}) + p_n(\mathbf{x})}, \quad \mathbf{g}(\mathbf{x}) = \nabla_{\theta} \ln p_m(\mathbf{x}; \theta)|_{\theta^*}$$

- Choice of the contrastive noise distribution: Some examples are a Gaussian or uniform distribution, a Gaussian mixture distribution, or an ICA distribution.

Formalized definition

- We have a multiclass problem where each training example $(\mathbf{x}_i, T_i)$ consists of a context $\mathbf{x}_i$, a small set of target classes T_i out of a large universe L of possible classes.
- We wish to learn a compatibility function $F(\mathbf{x}, y)$ which says something about the compatibility of a class y with a context $\mathbf{x}$.
- **Candidate Sampling** training methods involve constructing a training task in which for each training example $(\mathbf{x}_i, T_i)$, we only need to evaluate $F(\mathbf{x}, y)$ for a small set of candidate classes $C_i \subset L$, $L = \{T_i\}$.
Typically, the set of candidates C_i is the union of the target classes with a randomly chosen sample of (other) classes $S_i \subset L$:

$$C_i = T_i \cup S_i$$

The random choice of S_i may or may not depend on $\mathbf{x}_i$ and/or T_i .

Candidate Sampling Algorithms

	Positive training classes associated with training example (x_i, T_i) : $POS_i =$	Negative training classes associated with training example (x_i, T_i) : $NEG_i =$	Input to Training Loss $G(x, y) =$	Training Loss	$F(x, y)$ gets trained to approximate:
Noise Contrastive Estimation (NCE)	T_i	S_i	$F(x, y) - \log(Q(y x))$	Logistic	$\log(P(y x))$
Negative Sampling	T_i	S_i	$F(x, y)$	Logistic	$\log\left(\frac{P(y x)}{Q(y x)}\right)$
Sampled Logistic	T_i	$(S_i - T_i)$	$F(x, y) - \log(Q(y x))$	Logistic	$\log odds(y x) = \log\left(\frac{P(y x)}{1-P(y x)}\right)$
Full Logistic	T_i	$(L - T_i)$	$F(x, y)$	Logistic	$\log(odds(y x)) = \log\left(\frac{P(y x)}{1-P(y x)}\right)$

Candidate Sampling Algorithms

	Positive training classes associated with training example (x_i, T_i) : $POS_i =$	Negative training classes associated with training example (x_i, T_i) : $NEG_i =$	Input to Training Loss $G(x, y) =$	Training Loss	$F(x, y)$ gets trained to approximate:
Full Softmax	$T_i = \{t_i\}$	$(L - T_i)$	$F(x, y)$	Softmax	$\log(P(y x)) + K(x)$
Sampled Softmax	$T_i = \{t_i\}$	$(S_i - T_i)$	$F(x, y) - \log(Q(y x))$	Softmax	$\log(P(y x)) + K(x)$